

Chapter_07_b

September 7, 2025

0.1 # Chapter 7 (b) – Effective Agentic System Design Techniques (*continued*)

Install dependencies

```
[ ]: !pip install crewai langchain-openai langmem langgraph
```

```
[1]: import getpass
import os

api_key = getpass.getpass(prompt="Enter OpenAI API Key: ")
os.environ["OPENAI_API_KEY"] = api_key
```

```
[2]: from crewai import Agent, Task, Crew, Process
from crewai.tools import tool
from langchain_openai import ChatOpenAI
from IPython.display import display, Markdown, HTML

llm = ChatOpenAI(model="gpt-4o")
```

1 State spaces and environment modeling

In this section of Chapter 7, you read about some of the concepts of environment modeling and what Static and Dynamic environments are. These concepts ultimately tie back to how you maintain agent memory throughout an agentic workflow. So let's look at examples of the three different memory types we discussed-

- Short-term memory (working memory)
- Long-term memory (knowledge base)
- Episodic memory (interaction history)

Most agentic frameworks supports some sort of memory management (for example CrewAI's [memory management](#)), there are other frameworks that are solely purpose built for memory management in agentic systems, such as [LangMem](#).

1.0.1 Short-term memory

In this example we use LangGraph's thread-scoped memory. This is a type of memory that lets your application remember previous interactions within a single thread or conversation. A thread

organizes multiple interactions in a session, similar to the way email groups messages in a single conversation. In the following example, you will notice that we interact with the agent with two different thread_id and the agent is able to keep the memory separate for each thread rather than updating the global long-term memory.

```
[11]: from typing import TypedDict, List
      from langchain_core.messages import HumanMessage, AIMessage, SystemMessage, BaseMessage
      from langgraph.graph import StateGraph, START
      from langgraph.checkpoint.memory import MemorySaver
      from langchain_openai import ChatOpenAI

      # Define a simple state type for our travel agent
      class TravelState(TypedDict):
          messages: List[BaseMessage]

      # Define the travel agent response function
      def generate_response(state: TravelState) -> TravelState:
          """Generate a response based on the conversation history."""
          # Create a system message with the travel agent prompt
          system_message = SystemMessage(content="""
          You are a helpful travel agent assistant. Use the conversation history to
          remember the user's preferences and trip details. Be specific and reference
          their previously mentioned preferences when making recommendations.
          """)

          # Combine the system message with the existing messages
          messages = [system_message] + state["messages"]

          # Generate a response
          llm = ChatOpenAI(model="gpt-4o")
          response = llm.invoke(messages)

          # Return the updated state with the new message
          return {"messages": state["messages"] + [response]}

      # Build the graph
      builder = StateGraph(TravelState)
      builder.add_node("generate_response", generate_response)
      builder.add_edge(START, "generate_response")
      builder.set_finish_point("generate_response")

      # Compile the graph with a memory checkpointer
      checkpointer = MemorySaver()
      graph = builder.compile(checkpointer=checkpointer)

      # Function to process a user message
```

```

def chat_with_travel_agent(message: str, thread_id: str = "default"):
    """Process a user message and return the agent's response."""
    # Create the thread configuration
    config = {"configurable": {"thread_id": thread_id}}

    # Get the current state if it exists
    current_messages = []
    try:
        checkpoint = checkpointer.get_tuple(config)
        if checkpoint and "messages" in checkpoint.checkpoint:
            get("channel_values", {}):
                current_messages = checkpoint.
            checkpoint["channel_values"]["messages"]
        except:
            pass

    # Add the new message
    current_messages.append(HumanMessage(content=message))

    # Process through the graph
    result = graph.invoke({"messages": current_messages}, config)

    # Return just the last message content (the response)
    return result["messages"][-1].content

# Example usage
if __name__ == "__main__":
    thread_id = "user_123"

    # First interaction
    print("User: I want to plan a trip to Japan next month.")
    response = chat_with_travel_agent("I want to plan a trip to Japan next_
month.", thread_id)
    print(f"Agent: {response}\n")

    # Second interaction - the agent should remember Japan
    print("User: I'm interested in traditional culture and my budget is $3000.")
    response = chat_with_travel_agent("I'm interested in traditional culture_
and my budget is $3000.", thread_id)
    print(f"Agent: {response}\n")

    # Third interaction - test memory of previous details
    print("User: What was my destination again?")
    response = chat_with_travel_agent("What was my destination again?",
thread_id)
    print(f"Agent: {response}\n")

```

```

# New conversation thread (should not know about Japan)
new_thread = "user_456"
print("=== New Conversation ===")
print("User: What kind of budget would I need for a beach vacation?")
response = chat_with_travel_agent("What kind of budget would I need for a
↪beach vacation?", new_thread)
print(f"Agent: {response}")

```

User: I want to plan a trip to Japan next month.

Agent: Japan in November is beautiful with the autumn leaves in full display. Are there specific cities or experiences you're interested in, such as visiting Tokyo's vibrant neighborhoods, Kyoto's historic temples, or perhaps experiencing a traditional ryokan stay? Let me know your preferences, and I can make some recommendations!

User: I'm interested in traditional culture and my budget is \$3000.

Agent: With a focus on traditional culture and a budget of \$3000, you can have a memorable experience in Japan. Here's a suggested itinerary:

1. **Kyoto**:

- **Stay**: Consider staying in a ryokan (traditional inn) to immerse yourself in local culture. Tawaraya is a renowned option, although it might be on the higher end of your budget. For a more budget-friendly choice, try something like Hiiragiya Bekkan.

- **Activities**: Explore Kinkaku-ji (Golden Pavilion), Fushimi Inari Shrine, and the historic district of Gion to potentially spot geisha.

2. **Nara**:

- **Day Trip**: From Kyoto, take a day trip to Nara, known for its historical significance and the friendly deer that roam freely.

- **Sites**: Visit Todai-ji Temple with its Great Buddha, and Kasuga-taisha, a Shinto shrine with hundreds of lanterns.

3. **Kanazawa**:

- **Stay**: Enjoy the more relaxed atmosphere here, famous for its beautifully preserved Edo-era districts and samurai residences.

- **Activities**: Visit Kenroku-en, one of Japan's "Three Great Gardens," and explore the traditional teahouse district of Higashi Chaya.

4. **Transportation**:

- Consider getting a Japan Rail Pass to save on travel costs between cities.

5. **Cuisine**:

- Experience kaiseki, a traditional multi-course Japanese dinner, which is available in Kyoto and Kanazawa.

Your budget allows you to enjoy quality experiences and accommodations,

especially with careful planning. Let me know if you need more specific accommodation or activity suggestions!

User: What was my destination again?

Agent: You mentioned that you want to plan a trip to Japan next month with a focus on traditional culture, and a budget of \$3000.

=== New Conversation ===

User: What kind of budget would I need for a beach vacation?

Agent: The budget for a beach vacation can vary greatly depending on several factors, such as the destination, length of stay, type of accommodation, and activities you wish to engage in. Here's a general breakdown:

1. ****Destination****: Popular beach destinations like the Maldives, Hawaii, or Seychelles can be more expensive than beaches in Southeast Asia (like Thailand or Vietnam) or certain parts of the Caribbean.
2. ****Accommodation****:
 - Budget options like hostels or budget hotels start around \$20-50 per night.
 - Mid-range hotels or vacation rentals might range from \$75-200 per night.
 - Luxury resorts can be \$300+ per night.
3. ****Flights****: Depending on your departure location and destination, round-trip flights can range from \$200 to \$1,000+. Booking in advance or during sales can help reduce costs.
4. ****Food and Dining****:
 - Street food or casual dining can be as low as \$5-15 per meal.
 - Mid-range restaurants might charge \$20-50 per meal.
 - Fine dining could be \$75+ per meal.
5. ****Activities****: Beach activities like snorkeling, surfing lessons, or boat tours can range from \$20-100 per activity.
6. ****Transportation****: Consider local transportation like buses, taxis, or rentals. Costs can vary based on the region.
7. ****Miscellaneous****: Souvenirs, tips, and other unexpected expenses could add to the budget.

To better tailor recommendations, let me know if you have a specific destination in mind or any preferences regarding accommodation or activities!

1.0.2 Long-term Memory

In this example we will use LangMem to implement “memory collections” which is also known as long-term memory or knowledgebase. In this type, memories are stored as individual documents or records. For each new conversation, the memory system can decide to insert new mem-

ories to the store. In this case, a long-term memory is saved globally within a namespace named `travel_preferences`. Whenever we define the agentic workflow with that namespace, regardless of the thread it will refer to the globally stored preferences. This also can be useful to store information such as travel advisories, weather conditions etc. that apply globally to all travel related conversations and are perhaps not user specific.

```
[ ]: from langgraph.prebuilt import create_react_agent
from langgraph.store.memory import InMemoryStore
from langmem import create_manage_memory_tool, create_search_memory_tool
# from langchain_openai import ChatOpenAI

# Set up storage with vector embedding capabilities
store = InMemoryStore(
    index={
        "dims": 1536,
        "embed": "openai:text-embedding-3-small",
    }
)

# Custom tools for the travel agent
def search_flights(destination: str, dates: str, budget: str = None):
    """Search for flights based on destination, dates, and optional budget."""
    return f"Found several flight options to {destination} for {dates}. Prices range from $500-$1200 round trip."

def search_hotels(destination: str, dates: str, preferences: str = None):
    """Search for hotels based on destination, dates, and preferences."""
    return f"Found 15 hotels in {destination} for {dates}. Options include boutique hotels, chain hotels, and vacation rentals."

def search_activities(destination: str, interests: str = None):
    """Search for activities based on destination and interests."""
    return f"Popular activities in {destination} include museums, guided tours, outdoor activities, and local cuisine experiences."

# Create a travel agent with memory capabilities
travel_agent = create_react_agent(
    "gpt-4o",
    tools=[
        # Travel-specific tools
        search_flights,
        search_hotels,
        search_activities,

        # Memory management tools
        create_manage_memory_tool(
            namespace="travel_preferences",),
    ],
)
```

```

        instructions="""
        Proactively call this tool when you:
        1. Learn about a new user preference for travel (budget,
↪destinations, activities, diet, etc.)
        2. Receive an explicit request to remember specific trip details
        3. Need to record important context about their upcoming travel
↪plans
        4. Need to update incorrect or outdated information about the
↪user's travel preferences
        """
    ),
    create_search_memory_tool(
        namespace="travel_preferences",
    ),
],
store=store,
)

# Example function to handle travel planning conversation
def travel_planning_session(user_message, user_id="user123"):
    """
    Handle a user message in the travel planning conversation.

    Args:
        user_message: The user's message
        user_id: Unique identifier for the user

    Returns:
        The agent's response
    """
    # Set up the messages with the user's input
    messages = [{"role": "user", "content": user_message}]

    # Invoke the agent with the configured store for memory persistence
    response = travel_agent.invoke({"messages": messages})

    # Return the agent's response
    return response["messages"][-1].content

# Demonstrate usage with a typical travel planning conversation
if __name__ == "__main__":
    # First conversation establishing preferences
    print("User: I'm planning a trip to Barcelona in June for about a week. I
↪love architecture and food.")
    response = travel_planning_session("I'm planning a trip to Barcelona in
↪June for about a week. I love architecture and food.")
    print(f"Travel Agent: {response}\n")

```

```

# Second message with budget information
print("User: My budget is around $3000 for the entire trip, and I prefer_
↳staying in boutique hotels.")
response = travel_planning_session("My budget is around $3000 for the_
↳entire trip, and I prefer staying in boutique hotels.")
print(f"Travel Agent: {response}\n")

# Asking about vegetarian restaurants (new information)
print("User: I'm vegetarian. Can you recommend some good vegetarian_
↳restaurants in Barcelona?")
response = travel_planning_session("I'm vegetarian. Can you recommend some_
↳good vegetarian restaurants in Barcelona?")
print(f"Travel Agent: {response}\n")

# Testing memory recall - should remember destination, budget, dietary_
↳preferences
print("User: What was my budget again?")
response = travel_planning_session("What was my budget again?")
print(f"Travel Agent: {response}\n")

# Testing memory recall - should remember destination, interests
print("User: Can you suggest an architecture-focused itinerary?")
response = travel_planning_session("Can you suggest an architecture-focused_
↳itinerary?")
print(f"Travel Agent: {response}")

```

User: I'm planning a trip to Barcelona in June for about a week. I love architecture and food.

Travel Agent: In Barcelona, you'll find a plethora of activities and accommodations to suit your interests in architecture and food:

Activities:

- **Architecture Tours**: Discover Gaudí's masterpieces like the Sagrada Família and Park Güell.
- **Food Experiences**: Dive into local cuisine with tapas tours and cooking classes.

Hotels:

- **Accommodations**: There are numerous hotels, including boutique and chain hotels, as well as vacation rentals, conveniently located near architectural sites and popular food spots.

If you need more detailed information or personalized suggestions, feel free to ask!

User: My budget is around \$3000 for the entire trip, and I prefer staying in

boutique hotels.

Travel Agent: I've noted that your budget for the trip is around \$3000, and you prefer staying in boutique hotels. If there's anything else you'd like to plan or inquire about, feel free to let me know!

User: I'm vegetarian. Can you recommend some good vegetarian restaurants in Barcelona?

Travel Agent: While I couldn't find specific vegetarian restaurants directly through an activities search, Barcelona is known for its diverse culinary scene, including vegetarian and vegan options. Here are some popular vegetarian-friendly restaurants you might want to try:

1. ****Teresa Carles****: Known for its creative and delicious vegetarian dishes.
2. ****Flax & Kale****: Offers a variety of healthy vegetarian and vegan options.
3. ****Rasoterra****: A vegetarian bistro focusing on organic and local produce.
4. ****Veggie Garden****: Offers a mix of international vegetarian dishes.
5. ****Biocenter****: Serves a wide range of vegetarian dishes in a cozy atmosphere.

These restaurants are highly rated by locals and travelers alike. Enjoy your culinary exploration in Barcelona!

User: What was my budget again?

Travel Agent: Your budget for the trip is around \$3000, and you prefer staying in boutique hotels.

User: Can you suggest an architecture-focused itinerary?

Travel Agent: I can help you design an architecture-focused itinerary! To create a personalized plan, I'll need a few details:

1. Destination: Where are you planning to visit?
2. Duration: How many days will your trip last?
3. Interests: Are there specific architectural styles or landmarks you're interested in (e.g., modern, gothic, historical)?
4. Preferences: Do you have any preferences for your trip, such as budget, pace, or additional activities?
5. Dates: When are you planning to travel?

Let me know, and we can start crafting your itinerary!

Let's test the Agent's memory one more time by asking if it remembers the travel budget given to it earlier.

```
[ ]: print("User: I think I gave you a budget for my travel right?")
      response = travel_planning_session("I think I gave you a budget for my travel_
      ↪right?")
      print(f"Travel Agent: {response}\n")
```

User: I think I gave you a budget for my travel right?

Travel Agent: Yes, you mentioned that your budget for the trip is around \$3000, and you prefer staying in boutique hotels.

1.0.3 Episodic memory (interaction history)

Episodic memory preserves successful interactions as learning examples that guide future behavior. Unlike short-term memory which stores facts, episodic memory captures the full context of an interaction—the situation, the thought process that led to success, and why that approach worked. These memories help the agent learn from experience, adapting its responses based on what has worked before. In the following example, we create a travel agent system with episodic memory using the `langmem` library. The code defines a `TravelEpisode` schema that captures four critical elements:

- the customer’s travel request
- the agent’s planning considerations
- the specific recommendation provided, and
- the positive outcome that resulted.

When the system processes a conversation where a traveler expresses concerns about Italian crowds, it stores how the agent successfully redirected them to less-crowded alternatives while still satisfying their desire for art and cuisine. This structured approach to remembering successful interactions allows the system to develop a repository of effective strategies that can be applied to similar situations in the future.

```
[22]: from pydantic import BaseModel, Field
      from langmem import create_memory_manager

      class TravelEpisode(BaseModel):
          """An episode captures how to handle a specific travel planning situation,
          including the reasoning process
          and what made it successful."""

          customer_request: str = Field(
              ...,
              description="The travel request and relevant customer context"
          )
          considerations: str = Field(
              ...,
              description="Key travel planning considerations and reasoning process"
          )
          recommendation: str = Field(
              ...,
              description="What travel options were recommended in response"
          )
          outcome: str = Field(
              ...,
              description="How the customer responded and why the recommendation
              worked"
```

```

)

manager = create_memory_manager(
    "gpt-4o",
    schemas=[TravelEpisode],
    instructions="Extract examples of successful travel planning interactions.␣
↳Include the customer context, consideration process, and why the␣
↳recommendation satisfied the customer.",
    enable_inserts=True,
)

# Example conversation
conversation = [
    {"role": "user", "content": "I want to visit Italy in June but I'm worried␣
↳about crowds. I love art and food but hate waiting in long lines."},
    {"role": "assistant", "content": "Instead of Rome and Florence which get␣
↳extremely crowded in June, consider Bologna and Ravenna. Bologna offers␣
↳amazing food (it's nicknamed 'La Grassa' or 'The Fat One') and beautiful␣
↳medieval architecture without the overwhelming crowds. Ravenna has stunning␣
↳Byzantine mosaics and rarely has long lines. You could spend 4 days in␣
↳Bologna and 2 in Ravenna with a day trip to Modena for balsamic vinegar␣
↳tasting."},
    {"role": "user", "content": "That sounds perfect! I never considered those␣
↳cities but they seem to match exactly what I'm looking for. How would I get␣
↳between them?"},
]

episodes = manager.invoke({"messages": conversation})

```

```

[25]: for episode in episodes:
    print(f"{episode.content.customer_request=}")
    print(f"{episode.content.considerations=}")
    print(f"{episode.content.recommendation=}")
    print(f"{episode.content.outcome=}")

```

```

episode.content.customer_request="I want to visit Italy in June but I'm worried
about crowds. I love art and food but hate waiting in long lines."
episode.content.considerations='The customer wishes to visit Italy in June but
wants to avoid the large crowds typical of popular tourist destinations. They
have a strong interest in art and food but dislike waiting in long lines, which
means lesser-known cities with rich art and culinary offerings could be more
suitable than major tourist hubs like Rome or Florence.'
episode.content.recommendation="Visit Bologna and Ravenna instead of Rome and
Florence. Bologna is famous for its food and medieval architecture without the
intense tourist crowds. Ravenna is known for Byzantine mosaics and usually
doesn't have long lines. A suggested itinerary could be 4 days in Bologna, 2
days in Ravenna, with a day trip to Modena for balsamic vinegar tasting."

```

```
episode.content.outcome="The customer was very satisfied with the recommendation because it matched their love for art and food while addressing their concerns about crowds and long lines. They appreciated considering lesser-known cities they hadn't previously thought about."
```

1.0.4 Homework

Can you implement the same examples of long-term memory, short-term memory, and episodic memory using CrewAI's memory management? Refer to documentation for more details - <https://docs.crewai.com/concepts/memory>

2 Sequential and parallel processing in agentic workflows

The good news is that we have already performed parallel i.e. hierarchical workflows in the Chapter_06.ipynb notebook in the CWD approach. We used a Delegator Agent to delegate tasks to multiple worker agents.

As opposed to Hierarchical workflows, sequential workflows execute agents one after the other and are important in situations where the output of one agent is required for the next agent. For example- the flight booking dates need to be finalized by the flight booking agent before the hotel booking agent can book or recommend hotels. In CrewAI, a sequential workflow is specified by `Process.sequential` for example:

```
my_crew = Crew(
    agents=[agent_1, agent_2],
    tasks=[task_1, task_2],
    verbose=True,
    process=Process.sequential
)
```

2.0.1 Homework

Can you implement a sequential agentic workflow with the following example?

- Two agents a flight search and a hotel search agents
- Each agent uses its own tool
- The flight search agent must finalize the flights first with dates
- The hotel search agent will use the flight arrival and departure dates to recommend hotels, check-in and check-out dates